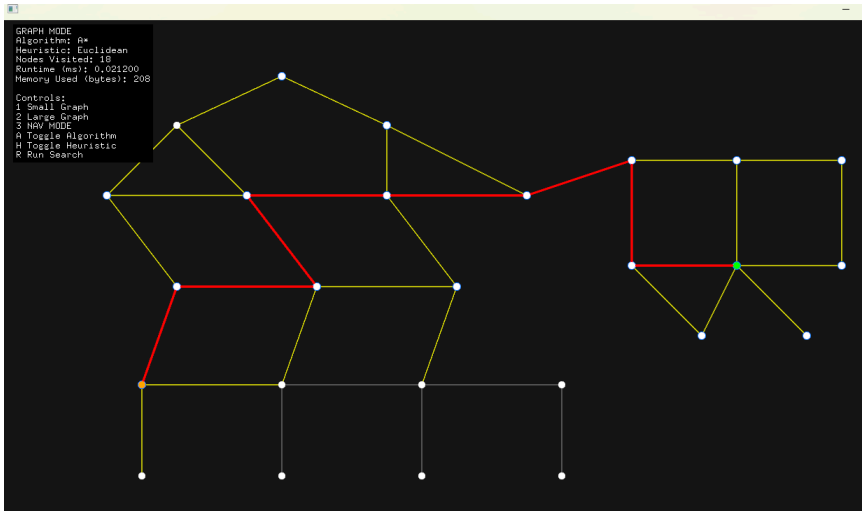
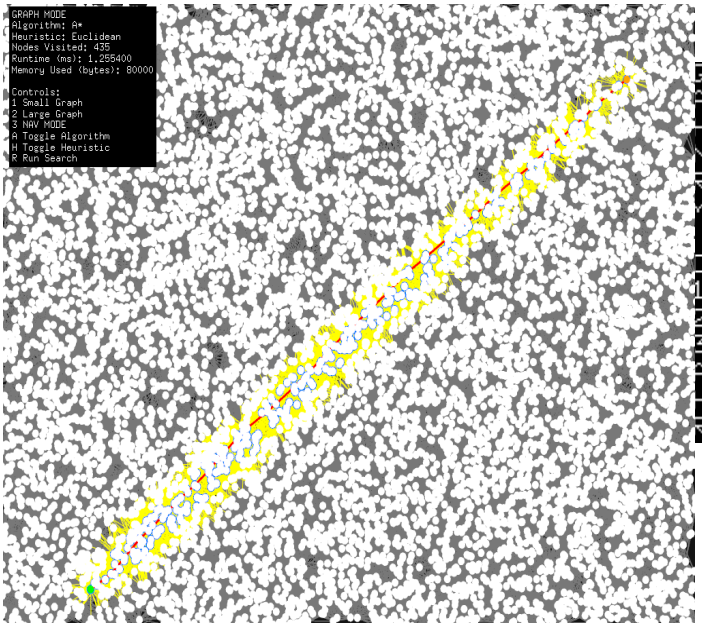
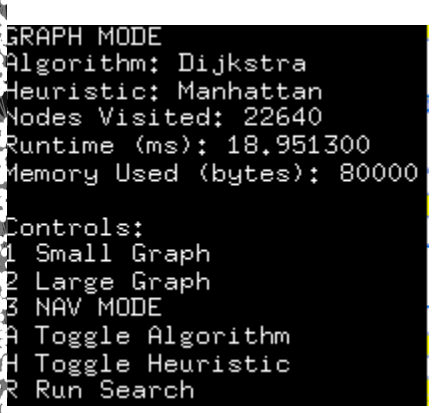
Pathfinding AI Process Letter

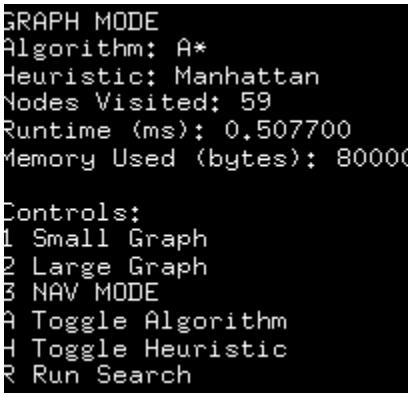
The process of completing this assignment involved again reading through the AI for Games Book and utilizing the pseudo code for creating a graph and the pathfinding algorithms. For a small graph, I designed a weighted directed graph representing the University of Utah Campus. Each node corresponds to a meaningful location, and edges represent walkable paths weighted by Euclidean distance. I intentionally kept the graph large enough to observe meaningful algorithmic behavior, but small enough to inspect node expansions and verify correctness visually. All edge weights are positive, and connections are explicitly defined to maintain control over structure and connectivity.



The large graph, I generated a large-scale graph containing tens of thousands of nodes. Nodes were randomly distributed across the 2D space and connected to nearby neighbors within a fixed radius. The dense connectivity and scale were designed specifically to stress-test the runtime, memory usage, and node expansion behavior of Dijkstra's and A* algorithms. I chose to visualize these graphs to showcase the power of each algorithm, but also, it looks Epic!





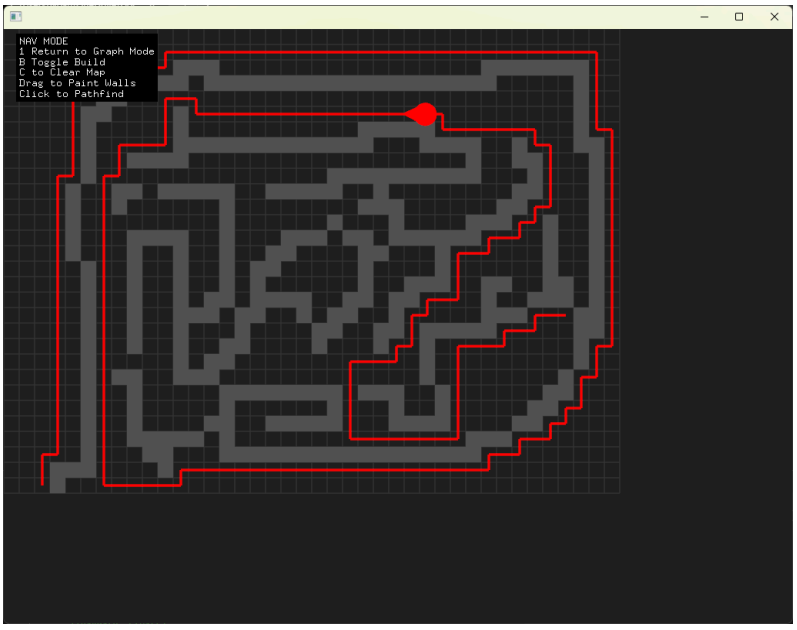


On the small graph, Dijkstra's algorithm expanded nodes uniformly outward from the start position, visiting a larger portion of the graph before reaching the goal. In contrast, A* with an admissible heuristic such as Manhattan or Euclidean distance significantly reduced the number of visited nodes by guiding the search toward the target. On the large graph, the performance gap became even more noticeable, as Dijkstra expanded a wide region while A* focused exploration in a directional manner.

One of the most interesting observations was how strongly graph structure influenced heuristic effectiveness. On spatially meaningful graphs, Manhattan and Euclidean heuristics provided clear guidance, drastically reducing node expansions. However, on the large random graph, heuristic quality had less impact because connectivity did not strictly align with geometric distance. This demonstrated that heuristic performance is highly dependent on how well the heuristic reflects the true cost structure of the graph.

Initially, I started with simple heuristics, such as constant and random, to observe baseline behavior, and I quickly saw how unoptimized A* became when the heuristic had no guidance. After adding Manhattan and Euclidean distance, I observed significantly fewer nodes visited as well as very fast completion times (especially compared to Dijkstra), as these heuristics better approximate true spatial cost and guide the search more efficiently toward the goal.

For the indoor navigation system, I implemented a uniform grid abstraction. Each grid cell represents a node in the navigation graph, and blocked cells represent walls. Edges connect only unblocked neighboring cells, making sure that computed paths never pass through obstacles.



When the user clicks anywhere in the window, the click position is quantized into a grid cell, and A* computes a path from the Boid's current cell to the target. The resulting node path is converted into world-space waypoints, which the Boid follows using the Seek behavior from Assignment 1. I wanted the system to feel interactive, so I also implemented a build mode that allows the player to manually place walls by dragging the mouse. These walls update the grid abstraction in real time, allowing the Boid to dynamically compute new paths around obstacles placed by the player. You can even clear the path and make a new one all at runtime.

```cpp
void ofApp::mousePressed(int x, int y, int button) {

    if (appMode_ == AppMode::NAV_MODE) {

        if (buildMode_) {
            navGrid_.blockCellAt(x, y);
            navGrid_.rebuild(navGraph_);
            return;
        }
        else {

            navStartNode_ = navGrid_.quantize(
                agent_.getPosition().x,
                agent_.getPosition().y);

            navGoalNode_ = navGrid_.quantize(x, y);

            if (navStartNode_ != -1 && navGoalNode_ != -1) {

                auto result = pathfinder_.aStar(
                    navGraph_,
                    navStartNode_,
                    navGoalNode_,
                    euclidean_);

                navPath_.clear();

                for (int id : result.path)
                    navPath_.push_back(navGrid_.nodeToWorld(id));

                agent_.setPath(navPath_);
            }
            return;
        }
    }
```

```cpp
class ManhattanHeuristic : public Heuristic {
public:
    float estimate(const Node& a, const Node& b) const override {
        return std::abs(a.position.x - b.position.x) +
            std::abs(a.position.y - b.position.y);
    }
};

class EuclideanHeuristic : public Heuristic {
public:
    float estimate(const Node& a, const Node& b) const override {
        return glm::distance(a.position, b.position);
    }
};
```

```cpp
void Boid::update(float dt, const NavGrid& grid) {

    if (currentTarget_ >= path_.size()) {
        velocity = { 0, 0 };
        return;
    }

    glm::vec2 target = path_[currentTarget_];
    glm::vec2 toTarget = target - position;
    float distance = glm::length(toTarget);

    // Switch waypoint quickly
    if (distance < 5.0f) {
        currentTarget_++;

        if (currentTarget_ >= path_.size()) {
            velocity = { 0, 0 };
            return;
        }

        target = path_[currentTarget_];
        toTarget = target - position;
    }

    // SEEK
    if (glm::length(toTarget) > 0.001f) {
        glm::vec2 desired = glm::normalize(toTarget) * maxSpeed;
        velocity = desired;
    }

    position += velocity * dt;

    if (glm::length(velocity) > 0.001f)
        orientation = atan2(velocity.y, velocity.x);
}
```